

CONTINUAL LEARNING OF DIFFUSION MODELS WITH GENERATIVE DISTILLATION

Sergi Masip*

Master in Computer Vision (Barcelona)
hello@sergimasip.com

Pau Rodríguez

Apple[†]
pau.rodriguez@apple.com

Tinne Tuytelaars

KU Leuven
tinne.tuytelaars@esat.kuleuven.be

Gido M. van de Ven

KU Leuven
gido.vandeven@kuleuven.be

ABSTRACT

Diffusion models are powerful generative models that achieve state-of-the-art performance in image synthesis. However, training them demands substantial amounts of data and computational resources. Continual learning would allow for incrementally learning new tasks and accumulating knowledge, thus enabling the reuse of trained models for further learning. One potentially suitable continual learning approach is generative replay, where a copy of a generative model trained on previous tasks produces synthetic data that are interleaved with data from the current task. However, standard generative replay applied to diffusion models results in a catastrophic loss in denoising capabilities. In this paper, we propose *generative distillation*, an approach that distills the entire reverse process of a diffusion model. We demonstrate that our approach substantially improves the continual learning performance of generative replay with only a modest increase in the computational costs.

1 INTRODUCTION

Diffusion models for likelihood-based data generation (Song & Ermon, 2019; Ho et al., 2020; Song et al., 2020b) have shown impressive results in tasks such as conditional and unconditional image generation. However, a disadvantage of these models is that they must be trained on vast datasets while using large amounts of computational resources, making their training very costly. Furthermore, once a diffusion model has been trained, incorporating new knowledge incrementally is challenging as the performance on previously learned tasks severely decreases when trained on new tasks. This phenomenon is known as catastrophic forgetting (McCloskey & Cohen, 1989; Ratcliff, 1990) and it is common when training neural networks in a sequential manner (De Lange et al., 2022). For this reason, deep learning practitioners typically retrain their models using all the data seen so far each time they incorporate new data. Especially for diffusion models, this is an extremely costly process. It would therefore be valuable to devise a way to leverage already trained diffusion models more efficiently and allow them to continuously expand their knowledge.

The continual learning field studies how to mitigate catastrophic forgetting to enable neural networks to incrementally learn new tasks, domains or classes while retaining previous knowledge (van de Ven et al., 2022). Among the existing approaches for continual learning, rehearsal methods, which store a fixed replay buffer of samples from previous tasks, have shown promising initial results for diffusion models (Zajac et al., 2023). However, previous data are sometimes unavailable or their use might be undesirable or even infeasible (e.g., due to privacy policies or computational cost). An alternative is generative replay (Shin et al., 2017), where the replay buffer is replaced with a generative model that produces synthetic samples. Previous works (Zajac et al., 2023; Gao & Liu, 2023) that used generative replay for continually training diffusion models employed the original sampler from “Denoising Diffusion Probabilistic Models” (DDPM) (Ho et al., 2020) to generate the synthetic samples.

An important limitation of DDPM is that its generation times are prohibitively slow. This restriction forced Zajac et al. (2023) and Gao & Liu (2023) to generate a fixed buffer of synthetic samples, thereby limiting the diversity advantage that diffusion models could offer. An alternative solution is to employ the sampler introduced in “Denoising Implicit Probabilistic Models” (DDIM) (Song et al., 2020a), which allows for much faster generation in exchange for a small reduction in image quality, enabling generative replay to use a considerably larger amount of samples. Nonetheless, we

* Work done under the Erasmus+ Traineeship program at KU Leuven.

† Work done externally as part of the Master in Computer Vision (Barcelona).

find that generative replay with the approximate samples yielded by DDIM results in a fast decrease in image quality, which seems to be due to a catastrophic loss in the model’s ability to denoise images from previous tasks. Previous works have observed severe forgetting with generative replay for diffusion models also when using DDPM (Smith et al., 2023; Gao & Liu, 2023). We hypothesize that the reason for this is that, in the case of diffusion models, the transfer of knowledge in standard generative replay is rather inefficient. As illustrated in Figure 1a, when training on a new task, standard generative replay transfers knowledge from the previous task model to the current task model only at the endpoint of the reverse process of the diffusion model (i.e., through the generated replay samples).

To improve generative replay for diffusion models, in this paper we propose to transfer knowledge more effectively by combining generative replay with knowledge distillation. This approach, which we call *generative distillation*, distills the entire reverse process of a *teacher* diffusion model trained on previous tasks to the *student* model being trained on the current task. The teacher generates a sample with DDIM, then Gaussian noise is added according to the variance schedule (Ho et al., 2020) and, finally, the objective matches the noise predictions of both the teacher and the student (Figure 1b). We conduct experiments on Fashion-MNIST (Xiao et al., 2017) and CIFAR-10 (Krizhevsky, 2009) where we continually train a diffusion model with generative distillation and demonstrate its effectiveness compared to generative replay at mitigating forgetting and the catastrophic loss in denoising capabilities, while increasing the computational cost by just one forward pass.

The main contributions of this paper can be summarized as follows:

- We show that continually training diffusion models using standard generative replay with DDIM results in a catastrophic loss in denoising capabilities (Section 4.1).
- To mitigate this issue, we introduce generative distillation, an approach that combines generative replay with knowledge distillation (Section 3.2).
- We demonstrate that generative distillation mitigates the loss in denoising capabilities and substantially improves upon generative replay, while using almost the same computational budget (Section 4.2).
- We further highlight the effectiveness of distillation for the continual training of diffusion models in additional experiments by applying it to input images of various quality, including Gaussian noise (Section 4.3).

2 BACKGROUND AND RELATED WORK

This section provides a brief description of methods and literature on diffusion models, distillation, and continual learning that are relevant to our work.

2.1 DIFFUSION MODELS

Diffusion models are likelihood-based models (Song & Ermon, 2019; Ho et al., 2020; Song et al., 2020b) defined by a forward process that adds noise to an image and a reverse process that denoises it. The forward process is a Markov chain with transitions that are Gaussian conditional distributions as defined by:

$$q(x_{1:T}|x_0) = \prod_{t=1}^T q(x_t|x_{t-1}), \quad q(x_t|x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t}x_{t-1}, \beta_t I). \quad (1)$$

This Markov chain progressively adds noise to an image x_0 through $x_1, x_2, \dots, x_T$ according to a variance schedule β_t . By setting $\alpha_t := 1 - \beta_t$ and $\bar{\alpha} := \prod_{s=1}^t \alpha_s$ so that $(1 - \bar{\alpha}_t)$ expresses the variance of the Gaussian noise, the above process can be reparameterized as:

$$q(x_t|x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)I), \quad (2)$$

allowing for sampling $x_t \sim q(x_t|x_0)$ at any timestep t in just one step.

The reverse process follows the reverse trajectory of the forward process. Although, in general, the shape of the posterior $q(x_{t-1}|x_t)$ is unknown, when $\beta_t \rightarrow 0$, it converges to a Gaussian (Sohl-Dickstein et al., 2015). Consequently, $q(x_{t-1}|x_t)$ can be approximated by modelling the mean and the variance of a conditional Gaussian distribution through a variational distribution $p_\theta(x_{t-1}|x_t)$. This gives the following reverse process:

$$p_\theta(x_{0:T}) = p(x_T) \prod_{t=1}^T p_\theta(x_{t-1}|x_t), \quad p_\theta(x_{t-1}|x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \Sigma_\theta(x_t, t)). \quad (3)$$

with θ the learnable parameters of the diffusion model.

A diffusion model, which from now on we will denote as ϵ_θ , can be trained to match the noise introduced during the forward process at any timestep t following a predefined variance schedule. This can be achieved by minimizing the following simplified objective (Ho et al., 2020):

$$L_{\text{simple}}(\theta) = \mathbb{E}_{t \sim \text{Unif}(T), x_0 \sim D, \epsilon \sim \mathcal{N}(0, I)} [\|\epsilon - \epsilon_\theta(\sqrt{\alpha_t}x_0 + \sqrt{1 - \alpha_t}\epsilon, t)\|^2], \quad (4)$$

where x_0 is a clean sample from the observed data distribution D , $\epsilon \sim \mathcal{N}(0, I)$ is the ground truth Gaussian noise injected during the forward pass, and α is a variance schedule function.

The standard way of sampling from a diffusion model (e.g., with DDPM) is computationally expensive as it requires many denoising steps (e.g., 1000 in Ho et al., 2020). This greatly limits the number of samples that can be generated for replay with a finite computational budget. An alternative way of sampling is provided by DDIM (Song et al., 2020a), which makes the forward process non-Markovian by reformulating $q(x_{t-1}|x_t, x_0)$ so that $q(x_t|x_{t-1})$ also depends on x_0 . DDIM also introduces a stochastic parameter σ ; when this parameter is set to zero, the sampling process becomes deterministic. DDIM permits sampling using a smaller number of denoising steps, thus allowing the generation of more images given the same computational budget. However, the fewer denoising steps are used, the blurrier and less detailed the sampled images tend to be. Each DDIM denoising step from $t \in T$ to $s \in T$ (such that $s < t$) can be expressed as:

$$x_s = \sqrt{\alpha_s} \left(\frac{x_t - \sqrt{1 - \alpha_t} \epsilon_\theta(x_t)}{\sqrt{\alpha_t}} \right) + \sqrt{1 - \alpha_s - \sigma_t^2} \cdot \epsilon_\theta(x_t) + \sigma_t \epsilon. \quad (5)$$

2.2 KNOWLEDGE DISTILLATION IN DIFFUSION MODELS

Existing studies use knowledge distillation in diffusion models to train a student network such that it can generate the same quality images as a teacher network, but in fewer generation steps (Luhman & Luhman, 2021; Salimans & Ho, 2021; Berthelot et al., 2023; Song et al., 2023; Gu et al., 2023). In this paper, we use distillation for a different purpose: we demonstrate its effectiveness for continual learning of diffusion models.

2.3 CONTINUAL LEARNING

An important problem in continual learning is catastrophic forgetting (McCloskey & Cohen, 1989; Ratcliff, 1990), a phenomenon where a sequentially trained neural network rapidly and drastically forgets its knowledge of previous tasks when it learns a new task. Current approaches that aim to mitigate catastrophic forgetting can be roughly categorized as architectural or parameter isolation methods, regularization methods and rehearsal methods (Biesialska et al., 2020; De Lange et al., 2022). Architectural methods (Rusu et al., 2016; Yoon et al., 2017; Scardapane et al., 2017; Masse et al., 2018; Wortsman et al., 2020; Iyer et al., 2022) prevent interference by introducing new modules or by using a fixed set of parameters for each task. Regularization methods (Kirkpatrick et al., 2017; Zenke et al., 2017; Li & Hoiem, 2017; Ahn et al., 2019) preserve the knowledge from previous tasks through the addition of extra terms to the loss function. Rehearsal methods (Ratcliff, 1990; Robins, 1995; Rebuffi et al., 2017; Lopez-Paz & Ranzato, 2017; Chaudhry et al., 2019) complement the training data of the current task with data representative of previous tasks.

Rehearsal is often implemented as experience replay, which uses an episodic buffer to store all or a subset of previous tasks' samples and replays them during the training of later tasks. In some situations, there may be privacy issues or compute constraints that forbid storing or using previous data. A popular approach to mitigate catastrophic forgetting under these circumstances is generative replay (Shin et al., 2017). Given a sequence of B tasks $\mathcal{T} = \{\mathcal{T}_1, \mathcal{T}_2, \dots, \mathcal{T}_B\}$, a generative model G_i is trained on data from task T_i interleaved with replay samples produced by G_{i-1} , a copy of the generative model after it finished training on the previous task.

2.4 CONTINUAL LEARNING AND DIFFUSION MODELS

The topic of continual learning of diffusion models remains relatively unexplored. We are aware of three works that use generative replay in diffusion models. Zajac et al. (2023) test different continual learning approaches on diffusion models, and Gao & Liu (2023) use a continually trained classifier to guide the generation process, as done by Dhariwal & Nichol (2021). These works, however, use the DDPM sampler and do not study how to improve generative replay for diffusion models through the addition of knowledge distillation. Both works further use task identity information in some way to guide the generation, whereas we do not inform our model about task identity and use an unconditional diffusion model. In other recent work, Smith et al. (2023) briefly report that they observe a rapid accumulation of errors when using standard generative replay, casting doubt on its efficacy in addressing catastrophic forgetting in diffusion models. To address this challenge, we propose a distillation-based approach that mitigates this issue and makes generative replay viable.

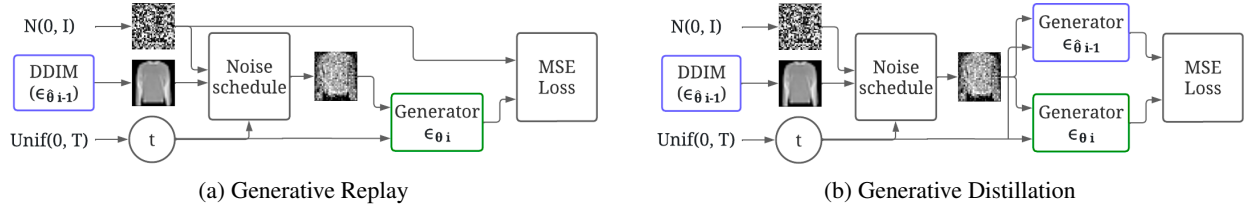


Figure 1: **Conceptual flow of generative replay and generative distillation.** In generative replay, the current task model ϵ_{θ_i} is trained to match the noise added to the replay sample, whereas, in generative distillation, ϵ_{θ_i} is trained to match the noise prediction of the previous task model $\epsilon_{\hat{\theta}_{i-1}}$.

3 METHODS

In this section, we first revisit standard generative replay for training diffusion models and explain how we implement it in our continual learning experiments. Second, we introduce our proposed approach, *generative distillation*, which combines generative replay with knowledge distillation to more effectively make use of the information contained in the previous task model.

3.1 STANDARD GENERATIVE REPLAY

In the continual learning setting we consider, a diffusion model ϵ_{θ} is incrementally trained on a sequence of tasks $\mathcal{T} = \{\mathcal{T}_1, \mathcal{T}_2, \dots, \mathcal{T}_B\}$. Before starting to train on a new task $\mathcal{T}_i$, we first store a copy of the diffusion model (i.e., a copy of the model as it is after finishing training on the previous task). We call this stored model copy the ‘previous task model’ or ‘teacher’, and denote it by $\epsilon_{\hat{\theta}_{i-1}}$. The model that is continued to be trained on the new task $\mathcal{T}_i$, we call the ‘current task model’ or ‘student’, and we denote it by ϵ_{θ_i} . (Note that the current task model ϵ_{θ_i} is initialized from the previous task model $\epsilon_{\hat{\theta}_{i-1}}$, as is common practice in continual learning literature.)

With standard generative replay, when the current task model ϵ_{θ_i} is trained on task $\mathcal{T}_i$, the previous task model $\epsilon_{\hat{\theta}_{i-1}}$ is used to generate replay samples which are combined with the training data of the current task. For every sample that is replayed, the previous task model $\epsilon_{\hat{\theta}_{i-1}}$ generates an image sample using DDIM, to which Gaussian noise $\epsilon \sim \mathcal{N}(0, I)$ is added to obtain a noisy sample x_t at timestep t :

$$x_t = \sqrt{\alpha_t} \cdot \text{DDIM}_N(\epsilon_{\hat{\theta}_{i-1}}) + \sqrt{1 - \alpha_t} \cdot \epsilon, \quad (6)$$

where $\text{DDIM}_N(\epsilon_{\hat{\theta}_{i-1}})$ is a clean sample generated by $\epsilon_{\hat{\theta}_{i-1}}$ using DDIM with N generation steps, and α is the noise schedule function. The loss on the replayed data is then:

$$L_{\text{replay}}(\theta_i) = \mathbb{E}_{t \sim \text{Unif}(T), x_t \sim \epsilon_{\hat{\theta}_{i-1}}} [\|\epsilon - \epsilon_{\theta_i}(x_t, t)\|_2^2]. \quad (7)$$

where ϵ is the noise added to the replayed image in Equation (6).

The final loss for generative replay is a weighted combination of the standard loss (Equation (4)) for data from the current task and the replay loss for replayed data:

$$L_{\text{GR}} = rL_{\text{simple}} + (1 - r)L_{\text{replay}}, \quad (8)$$

where r is a ratio that gives the loss on the current task a decreasing proportional importance relative to the number of tasks seen so far (i.e., $r = \frac{1}{i}$ with i the number of tasks learned so far) (van de Ven et al., 2020).

3.2 GENERATIVE DISTILLATION

In standard generative replay, the previous task model $\epsilon_{\hat{\theta}_{i-1}}$ is used in the training process of the current task model ϵ_{θ_i} only for sampling images for replay (Figure 1a, see Section 3.1 for details). This means that during training, knowledge from $\epsilon_{\hat{\theta}_{i-1}}$ is transferred to ϵ_{θ_i} only at the endpoint of the reverse process of the diffusion model, i.e. after going through all the denoising steps. Therefore, if the previous task model generates imperfect images (e.g., using a relatively small number of DDIM steps), the current task model might learn trajectories that lead to these images, causing an accumulation of errors. To tackle this issue, we introduce knowledge distillation into generative replay for diffusion models. Our approach, *generative distillation*, transfers the knowledge from $\epsilon_{\hat{\theta}_{i-1}}$ to ϵ_{θ_i} at each point of the reverse trajectory, while using synthetic images sampled from $\epsilon_{\hat{\theta}_{i-1}}$ as inputs.

Algorithm 1 Generative distillation

Input: Training data current task $\mathcal{D}_i$, previous task model $\epsilon_{\hat{\theta}_{i-1}}$, current task model ϵ_{θ_i} , number of generation steps N , and weighting parameter λ .

```

1: for  $x_0 \in \mathcal{D}_i$  do
2:   Sample  $t \sim \text{Unif}(1, \dots, T)$ ,  $s \sim \text{Unif}(1, \dots, T)$ 
3:   Sample  $\epsilon \sim \mathcal{N}(0, I)$ ,  $\bar{\epsilon} \sim \mathcal{N}(0, I)$ 
4:    $x_t = \sqrt{\alpha_t}x_0 + \sqrt{1 - \alpha_t}\epsilon$ 
5:    $\bar{x}_s = \sqrt{\alpha_s} \cdot \text{DDIM}_N(\epsilon_{\hat{\theta}_{i-1}}) + \sqrt{1 - \alpha_s}\bar{\epsilon}$ 
6:   Take gradient descent on
7:      $\nabla_{\theta_i} \left[ \frac{1}{i} \|\epsilon - \epsilon_{\theta_i}(x_t, t)\|_2^2 + (1 - \frac{1}{i})\lambda \|\epsilon_{\hat{\theta}_{i-1}}(\bar{x}_s, s) - \epsilon_{\theta_i}(\bar{x}_s, s)\|_2^2 \right]$ 
8: end for

```

With generative distillation, similar as with generative replay, the previous task model $\epsilon_{\hat{\theta}_{i-1}}$ generates first an image sample using DDIM and, then, Gaussian noise is added to obtain a noisy sample x_t at timestep t as in Equation (6). Given the noisy sample x_t , the distillation objective for incorporating knowledge from previous tasks during generative distillation is the following:

$$L_{\text{distil}}(\theta_i) = \mathbb{E}_{t \sim \text{Unif}(T), x_t \sim \epsilon_{\hat{\theta}_{i-1}}} [\|\epsilon_{\hat{\theta}_{i-1}}(x_t, t) - \epsilon_{\theta_i}(x_t, t)\|_2^2]. \quad (9)$$

In Figure 1b, a conceptual flow of generative distillation is shown. The final loss that we aim to minimize throughout the continual learning process is a weighted combination of the standard loss (Equation (4)) for data from the current task and the distillation loss for replayed data:

$$L_{\text{GD}} = rL_{\text{simple}} + (1 - r)\lambda L_{\text{distil}}, \quad (10)$$

where r is the same task importance ratio as in Equation (8), and λ is a weighting parameter to balance the magnitudes of the standard and distillation losses.

4 EXPERIMENTS

To compare the effectiveness of standard generative replay and our proposed generative distillation approach for continually training a diffusion model, we perform experiments using the Fashion-MNIST (Xiao et al., 2017) and CIFAR-10 (Krizhevsky, 2009) datasets. We divide both datasets into five tasks, each comprising two randomly selected classes (Figure 3a), that must be learned one after the other. For both task sequences, the diffusion model that we train is based on the U-Net architecture used for CIFAR-10 in Ho et al. (2020) (refer to Appendix A.1.1 for details). When using generative replay or generative distillation, we use the previous task model $\epsilon_{\hat{\theta}_{i-1}}$ to generate new samples to replay at each training step. We always generate as many replay samples as there are samples from the current task. To generate these replay samples, we use 2 DDIM steps on Fashion-MNIST and 10 DDIM steps on CIFAR-10. We base this choice on samples generated with a model trained on an independently and identically distributed stream of data using different amounts of DDIM steps (refer to Appendix B.1). For each dataset, we select the smallest number of DDIM steps that provide distinguishable objects. For further details on the experiments, models and metrics, please refer to Appendix A.

In the following, we mainly run experiments on the strategies explained in the previous section and baselines:

- **Generative replay:** the previous task model generates replay samples for the current task model.
- **Generative distillation:** the previous task model generates replay samples *and* transfers knowledge through distillation to the current task model.
- **Joint:** a model trained on data from all tasks, serving as our upper target.
- **Finetune:** a model naïvely fine-tuned on the current task data, serving as our lower target.

4.1 GENERATIVE REPLAY RESULTS IN A CATASTROPHIC LOSS IN DENOISING CAPABILITY

The images generated by a diffusion model are imperfect approximations of the true data distribution that present some error, even if they look decent and are distinguishable to the human eye. As shown in Figure 2, generative replay causes the error to add up, leading to a steep and steady degradation of the generation quality. The images generated by the diffusion model after training with generative replay become smooth and poorly detailed. According to Deja et al.

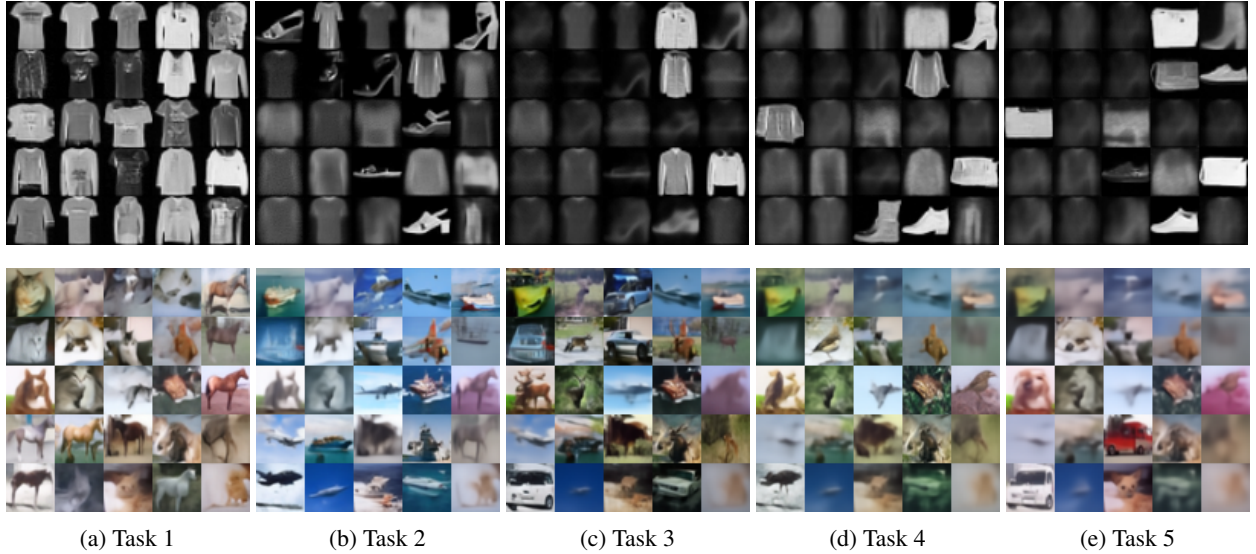


Figure 2: **Samples for generative replay on Fashion-MNIST (first row) and CIFAR-10 (second row).** To generate samples for previous tasks, the teacher used 2 DDIM steps on Fashion-MNIST and 10 DDIM steps on CIFAR-10. Standard generative replay causes a catastrophic accumulation of error for samples from previous tasks.

(2022), diffusion models can be decomposed in two parts: (i) a generative part that generates an image from corrupted noise, and (ii) a denoising part that seems responsible for also introducing new details to the image. In the continuous learning process, the model seems to maintain its generative capabilities, at least to some extent. For instance, when examining the second column in the last row of Fashion-MNIST in Figure 2, the model appears to preserve the t-shirt silhouette despite an increase in noise and a severe loss of finer details. Similarly, in CIFAR-10, the last sample in the final row exhibits a comparable behavior, where, in the last task (refer to Figure 2e), only an extremely blurry version of the original cat sample remains. These observations suggest that in particular the denoising capabilities of a diffusion model are susceptible to the shortcomings of generative replay.

4.2 GENERATIVE DISTILLATION SUBSTANTIALLY IMPROVES UPON GENERATIVE REPLAY

In the previous subsection, we observed that standard generative replay with a limited number of DDIM steps causes a severe degradation of the diffusion model’s denoising capabilities. Now we explore whether, without increasing the number of DDIM steps, this challenge can be overcome by our proposed generative distillation approach.

4.2.1 QUALITATIVE COMPARISON

In Figure 3, we show images sampled from the diffusion model after training on all tasks. While the model trained with generative replay generated smooth and blurry images, the model trained with generative distillation was still able to produce detailed images on both datasets. On top of that, the samples from generative distillation are visually comparable to those obtained after joint training, that is, after sequentially training the diffusion model using all the data from tasks so far. This demonstrates the effectiveness of generative distillation in preserving past knowledge.

4.2.2 QUANTITATIVE COMPARISON

To quantitatively compare the trained diffusion models, we report the commonly used Fréchet Inception Distance (FID) (Heusel et al., 2017). We also report the Kullback-Leibler divergence (KLD) between the ground truth class frequency distribution and the distribution of predicted class membership for images generated by the diffusion model. The KLD evaluates to what extent the diffusion model generates samples from all classes seen so far (rather than, for example, only from classes of the last task). When computing these evaluation metrics, we sample images using 10 DDIM steps on Fashion-MNIST and 20 DDIM steps on CIFAR-10. See Appendix A.5 for details.

The results in Table 1 and Figure 4 back up the observations made in the qualitative assessment. For example, the use of generative distillation resulted in $9.01\times$ lower FID on Fashion MNIST and $2.28\times$ lower FID on CIFAR-10 compared to generative replay. Similar results were observed for the KLD, which is $13.56\times$ lower on Fashion MNIST

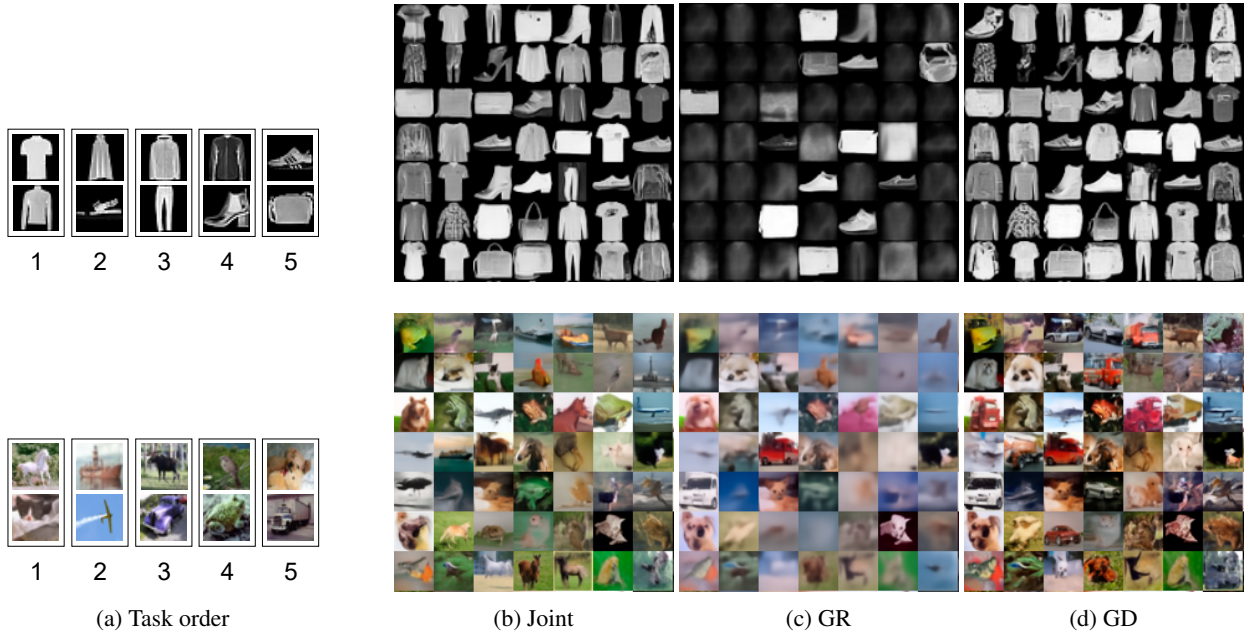


Figure 3: **Qualitative comparison of generative replay and generative distillation.** The first row corresponds to results on Fashion-MNIST and the second row on CIFAR-10. (a) The order of the tasks the models were trained on. (b-d) Images generated at the end of training by a diffusion model jointly trained on all tasks (Joint), continually trained using generative replay (GR) or continually trained using generative distillation (GD). Introducing distillation into generative replay prevents the catastrophic accumulation of error and the model is still able to produce detailed images.

Table 1: **Quantitative comparison of generative replay and generative distillation.** These quantitative results confirm that introducing distillation into generative replay substantially improves the continual learning performance of diffusion models. The FID and KLD are computed after training on all tasks. Reported for each metric is mean $\pm$ standard error over 3 random seeds.

Strategy	Fashion MNIST		CIFAR-10	
	$\downarrow$ FID	$\downarrow$ KLD	$\downarrow$ FID	$\downarrow$ KLD
Joint (upper target)	15.5 ± 1.3	0.08 ± 0.02	31.9 ± 2.9	0.12 ± 0.02
Finetune (lower target)	68.8 ± 7.0	4.82 ± 1.42	55.8 ± 2.6	1.14 ± 0.10
Generative replay	211.8 ± 6.9	3.39 ± 0.30	96.4 ± 5.3	0.74 ± 0.14
Generative distillation	23.5 ± 1.1	0.25 ± 0.10	42.3 ± 4.7	0.15 ± 0.01

and $4.93\times$ lower on CIFAR-10. These results show that generative distillation preserves the diversity of the diffusion model, achieving a KLD close to joint training. In the Appendix, we further show that generative distillation clearly outperforms generative replay also when increasing or decreasing the number of DDIM teacher steps (Figures B.5 to B.10 and B.17 to B.20). In Appendix B.2 we further compare the computational and memory costs of generative distillation and generative replay, and in Appendix B.3 we compare generative distillation with experience replay using various buffer sizes.

4.2.3 EVALUATION WITH A CONTINUALLY TRAINED CLASSIFIER

As an additional way of evaluating the performance of the continually trained diffusion model, we include a classifier trained with generative replay using the continually trained diffusion model as the generator (see Appendix A.2 for details). When generating samples to train the classifier, we used 10 DDIM steps on Fashion-MNIST and 20 DDIM steps on CIFAR-10. In Table 2, we report the accuracy of the classifier at the end of training. In line with the results discussed in previous sections, the model trained with generative distillations proved to generate samples that are more useful for training the classifier in both datasets. The low accuracy achieved on CIFAR-10 may be explained by the

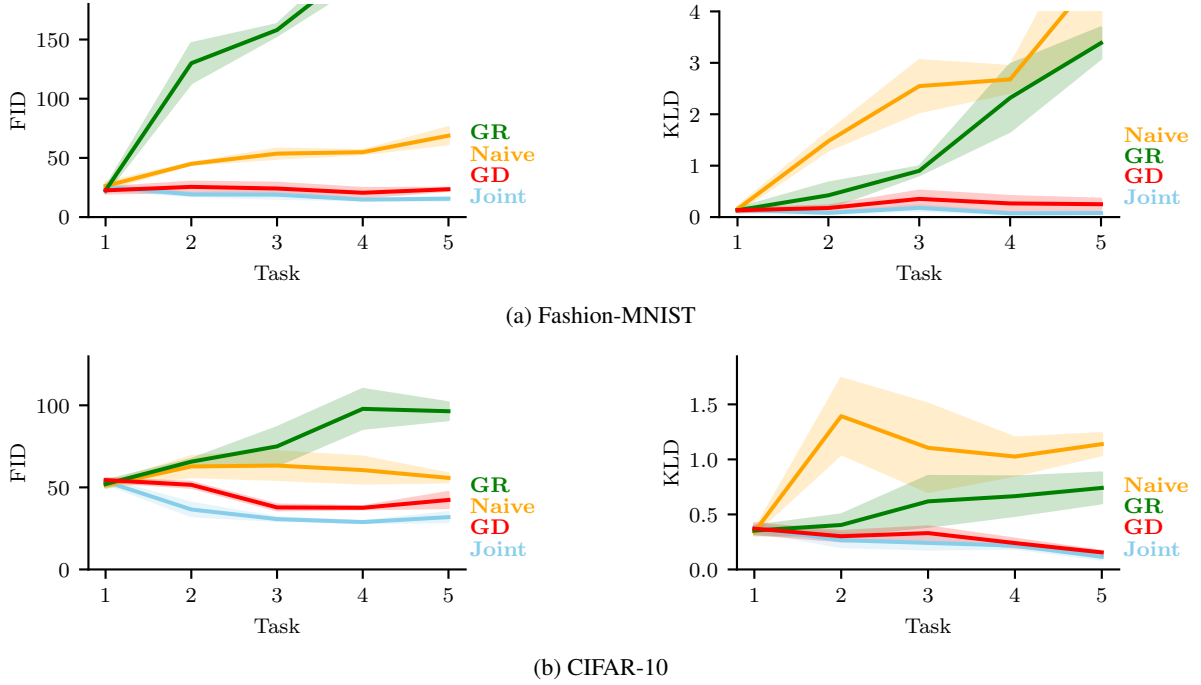


Figure 4: **Quality metrics of the diffusion model throughout the continual learning process.** For each approach, the FID and KLD of the diffusion model are computed after every task. Displayed are the means $\pm$ standard errors over 3 random seeds. Joint: training using data from all tasks so far (upper target), Naive: continual fine-tuning (lower target), GR: generative replay, GD: generative distillation.

Table 2: **Evaluation with a continually trained classifier.** The classifier is trained on the data of the current task along with replay samples from the continually trained diffusion model. Reported is mean $\pm$ standard error over 3 seeds.

Strategy	Fashion-MNIST $\uparrow$ ACC	CIFAR-10 $\uparrow$ ACC
Joint (<i>upper target</i>)	0.92 ± 0.00	0.77 ± 0.03
Finetune (<i>lower target</i>)	0.17 ± 0.03	0.19 ± 0.00
Generative replay	0.50 ± 0.04	0.26 ± 0.01
Generative distillation	0.72 ± 0.03	0.31 ± 0.03

higher complexity of the dataset and the relatively small amount of generation steps used to produce the samples. Nevertheless, this evaluation confirms the benefit of incorporating distillation into generative replay.

4.3 INFLUENCE OF THE QUALITY OF THE REPLAYED INPUTS ON GENERATIVE DISTILLATION

We studied the effect of increasing the number of DDIM steps used by the previous task model (teacher steps) in generative replay and generative distillation. As shown in Figure 5, increasing the number of teacher steps in generative replay alleviated the catastrophic loss in denoising capabilities to some extent. However, generative replay continued to be consistently outperformed by generative distillation, even when using 100 teacher steps. Using 100 teacher steps in generative distillation did not seem to provide significant benefits over using 10 teacher steps, despite the greater computational costs.

We also compare generative distillation with other forms of distillation. One alternative we implemented distills the knowledge of the model from the previous task using the current task’s data. This can be seen as a Learning without Forgetting (LwF) (Li & Hoiem, 2017) like approach applied to diffusion models. In Figure 6c, we can see that this form of distillation exhibits better performance than standard generative replay and mitigates the extent of the catastrophic loss in denoising capabilities. However, the overall quality of the samples degrades steadily and more artefacts show up

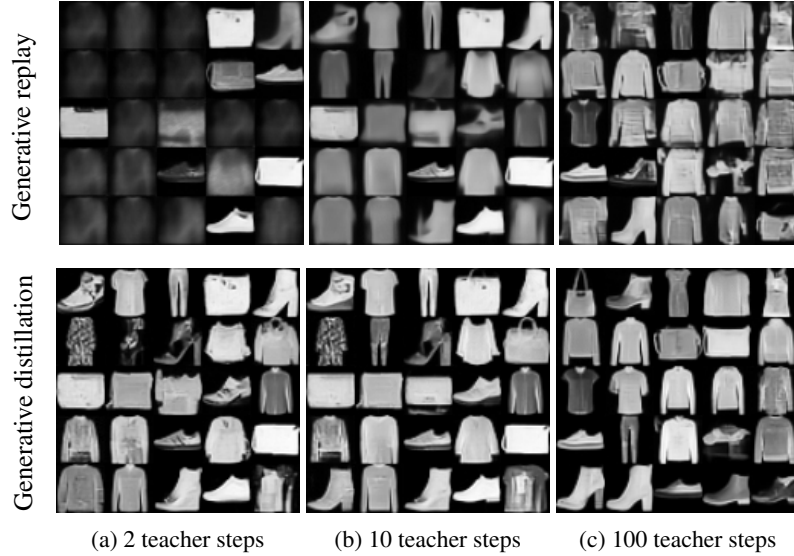


Figure 5: **Samples for different amounts of teacher steps on Fashion-MNIST.** Generative distillation continues to outperform generative replay when the number of DDIM steps to generate replay samples for previous tasks is varied. For a quantitative evaluation, see Table B.4 in the Appendix.

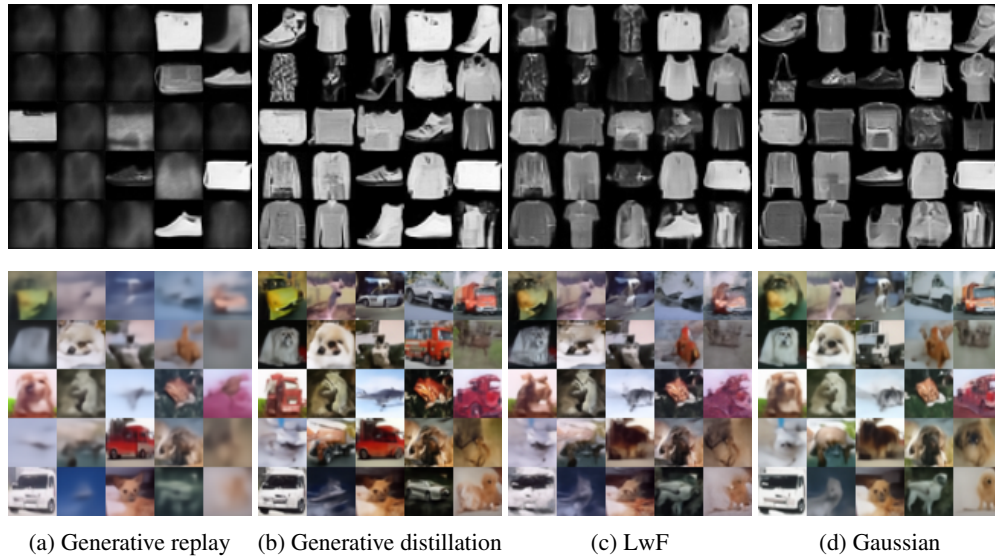


Figure 6: **Samples for different distillation strategies on Fashion-MNIST (first row) and CIFAR-10 (second row).** To generate samples for previous tasks, the teacher used 2 DDIM steps on Fashion-MNIST and 10 DDIM steps on CIFAR-10. Distilling with current data or Gaussian noise mitigates the catastrophic loss in denoising capabilities.

as more tasks are learned (e.g., the sample in the fourth column of the last row of Fashion-MNIST or the first sample in the last row of CIFAR-10).

Distilling with Gaussian noise as input for all timesteps is another distillation approach we considered, which is an extreme case equivalent to the model from the previous task using 0 DDIM steps. Strikingly, this approach also outperforms generative replay as observed in Figure 6d. Moreover, this strategy does not introduce as many artifacts as using current data (see the first sample in the last row of CIFAR-10). However, both approaches suffer a loss in diversity and tend to generate more samples from the current task, which could stem from a potential misalignment between the inputs and the true distribution of images.

5 LIMITATIONS AND FUTURE WORK

One limitation of this work involves the way we searched for the lambda hyperparameter that balances the magnitudes of the standard loss and the distillation loss in Equation (10). The sweep that we performed for this hyperparameter is not part of the continual learning process, since we trained on the entire task sequence several times and selected the best one at the end (see Appendix A.4). However, it is reassuring that the lambda values that we selected for Fashion MNIST also result in a strong performance for CIFAR-10.

Another factor to consider is that both generative replay and generative distillation introduce a significant computational overhead compared to finetuning, primarily due to the need for sample generation. Related to this, due to our limited time and computational resources, we used a number of training iterations that could be considered relatively small for diffusion models. For example, while in Ho et al. (2020) their models take 800k gradient updates on CIFAR-10, in our continual learning experiments, the models went through a total of 157k updates after training on all tasks.

Restrictions on available resources also informed our choice to use relatively small-scale image datasets (Fashion MNIST and CIFAR-10) for the experiments. By using these datasets we were able to perform extensive and carefully controlled comparisons. Performing such controlled continual learning experiments with diffusion models on larger-scale benchmarks would not have been feasible for us. Given the consistent and substantial benefit that we found for generative distillation, combined with our conceptual arguments about the benefits of this approach, we expect that generative distillation will also offer benefits when used in larger-scale settings. This however remains to be demonstrated, which is something we leave for future work.

Although in this work we focused on using generative distillation with DDIM, we note that our approach can also be combined with other, newer samplers for diffusion models (e.g., Lu et al., 2022a;b; Song et al., 2023). Such samplers might be able to allow generating samples for generative distillation with fewer teacher steps while still providing a good enough approximation of the true sample distribution. This would be particularly useful for more complex datasets, as we have observed that more teacher steps might be needed as the complexity of the dataset increases.

6 CONCLUSIONS

Continually training a diffusion model with standard generative replay results in a catastrophic loss in its denoising capabilities for past tasks, leading the model to produce blurry images. To address this problem, we propose including knowledge distillation into the generative replay process (generative distillation). Our experiments on Fashion-MNIST and CIFAR-10 show that generative distillation mitigates this catastrophic forgetting and markedly enhances the performance of the continually trained diffusion model. These results demonstrate the feasibility of continual learning in diffusion models, paving the way for future research into more robust, adaptable, and efficient generative models.

ACKNOWLEDGEMENTS

This project has been supported by funding from the European Union under the Erasmus+ Traineeship program, under the Horizon 2020 research and innovation program (ERC project KeepOnLearning, grant agreement No. 101021347) and under Horizon Europe (Marie Skłodowska-Curie fellowship, grant agreement No. 101067759).

REFERENCES

Hongjoon Ahn, Sungmin Cha, Donggyu Lee, and Taesup Moon. Uncertainty-based continual learning with adaptive regularization. *Advances in Neural Information Processing Systems*, 32, 2019.

- David Berthelot, Arnaud Autef, Jierui Lin, Dian Ang Yap, Shuangfei Zhai, Siyuan Hu, Daniel Zheng, Walter Talbot, and Eric Gu. TRACT: Denoising diffusion models with transitive closure time-distillation. *arXiv preprint arXiv:2303.04248*, 2023.
- Magdalena Biesialska, Katarzyna Biesialska, and Marta R. Costa-jussà. Continual lifelong learning in natural language processing: A survey. In *Proceedings of the 28th International Conference on Computational Linguistics*, pp. 6523–6541, Barcelona, Spain (Online), Dec 2020. International Committee on Computational Linguistics. doi: 10.18653/v1/2020.coling-main.574. URL <https://www.aclweb.org/anthology/2020.coling-main.574>.
- Arslan Chaudhry, Marc’Aurelio Ranzato, Marcus Rohrbach, and Mohamed Elhoseiny. Efficient lifelong learning with A-GEM. In *International Conference on Learning Representations*, 2019. URL https://openreview.net/forum?id=Hkf2_sC5FX.
- Matthias De Lange, Rahaf Aljundi, Marc Masana, Sarah Parisot, Xu Jia, Aleš Leonardis, Gregory Slabaugh, and Tinne Tuytelaars. A continual learning survey: Defying forgetting in classification tasks. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(7):3366–3385, 2022. doi: 10.1109/TPAMI.2021.3057446.
- Kamil Deja, Anna Kuzina, Tomasz Trzcinski, and Jakub Tomczak. On analyzing generative and denoising capabilities of diffusion-based deep generative models. *Advances in Neural Information Processing Systems*, 35:26218–26229, 2022.
- Prafulla Dhariwal and Alexander Nichol. Diffusion models beat GANs on image synthesis. *Advances in Neural Information Processing Systems*, 34:8780–8794, 2021.
- Rui Gao and Weiwei Liu. DDGR: Continual learning with deep diffusion-based generative replay. In *Proceedings of the 40th International Conference on Machine Learning*, 2023.
- Jiatao Gu, Shuangfei Zhai, Yizhe Zhang, Lingjie Liu, and Joshua M Susskind. BOOT: Data-free distillation of denoising diffusion models with bootstrapping. In *ICML 2023 Workshop on Structured Probabilistic Inference & Generative Modeling*, 2023.
- Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 770–778, 2016.
- Martin Heusel, Hubert Ramsauer, Thomas Unterthiner, Bernhard Nessler, and Sepp Hochreiter. GANs trained by a two time-scale update rule converge to a local nash equilibrium. *Advances in Neural Information Processing Systems*, 30, 2017.
- Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in Neural Information Processing Systems*, 33:6840–6851, 2020.
- Abhiram Iyer, Karan Grewal, Akash Velu, Lucas Oliveira Souza, Jeremy Forest, and Subutai Ahmad. Avoiding catastrophe: Active dendrites enable multi-task learning in dynamic environments. *Frontiers in Neurorobotics*, 16, 2022. ISSN 1662-5218. doi: 10.3389/fnbot.2022.846219. URL <https://www.frontiersin.org/articles/10.3389/fnbot.2022.846219>.
- Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, Demis Hassabis, Claudia Clopath, Dharshan Kumaran, and Raia Hadsell. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, Mar 2017. doi: 10.1073/pnas.1611835114. URL <https://doi.org/10.1073%2Fpnas.1611835114>.
- Alex Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009.
- Zhizhong Li and Derek Hoiem. Learning without forgetting. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 40(12):2935–2947, 2017.

- Vincenzo Lomonaco, Lorenzo Pellegrini, Andrea Cossu, Antonio Carta, Gabriele Graffieti, Tyler L. Hayes, Matthias De Lange, Marc Masana, Jary Pomponi, Guido van de Ven, Martin Mundt, Qi She, Keiland Cooper, Jeremy Forest, Eden Belouadah, Simone Calderara, German I. Parisi, Fabio Cuzzolin, Andreas Toliás, Simone Scardapane, Luca Antiga, Subutai Amhad, Adrian Popescu, Christopher Kanan, Joost van de Weijer, Tinne Tuytelaars, Davide Bacciu, and Davide Maltoni. Avalanche: an end-to-end library for continual learning. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops*, 2nd Continual Learning in Computer Vision Workshop, 2021.
- David Lopez-Paz and Marc’ Aurelio Ranzato. Gradient episodic memory for continual learning. *Advances in Neural Information Processing Systems*, 30, 2017.
- Cheng Lu, Yuhao Zhou, Fan Bao, Jianfei Chen, Chongxuan Li, and Jun Zhu. DPM-solver: A fast ODE solver for diffusion probabilistic model sampling in around 10 steps. *Advances in Neural Information Processing Systems*, 35: 5775–5787, 2022a.
- Cheng Lu, Yuhao Zhou, Fan Bao, Jianfei Chen, Chongxuan Li, and Jun Zhu. DPM-solver++: Fast solver for guided sampling of diffusion probabilistic models. *arXiv preprint arXiv:2211.01095*, 2022b.
- Eric Luhman and Troy Luhman. Knowledge distillation in iterative generative models for improved sampling speed. *arXiv preprint arXiv:2101.02388*, 2021.
- Nicolas Y. Masse, Gregory D. Grant, and David J. Freedman. Alleviating catastrophic forgetting using context-dependent gating and synaptic stabilization. *Proceedings of the National Academy of Sciences*, 115(44), oct 2018. doi: 10.1073/pnas.1803839115. URL <https://doi.org/10.1073%2Fpnas.1803839115>.
- Michael McCloskey and Neal J Cohen. Catastrophic interference in connectionist networks: The sequential learning problem. In *Psychology of Learning and Motivation*, volume 24, pp. 109–165. Elsevier, 1989.
- Roger Ratcliff. Connectionist models of recognition memory: constraints imposed by learning and forgetting functions. *Psychological Review*, 97:285–308, 1990.
- Sylvestre-Alvise Rebuffi, Alexander Kolesnikov, Georg Sperl, and Christoph H Lampert. icarl: Incremental classifier and representation learning. In *Proceedings of the IEEE conference on Computer Vision and Pattern Recognition*, pp. 2001–2010, 2017.
- Anthony Robins. Catastrophic forgetting, rehearsal and pseudorehearsal. *Connection Science*, 7(2):123–146, 1995. doi: 10.1080/09540099550039318. URL <https://doi.org/10.1080/09540099550039318>.
- Andrei A Rusu, Neil C Rabinowitz, Guillaume Desjardins, Hubert Soyer, James Kirkpatrick, Koray Kavukcuoglu, Razvan Pascanu, and Raia Hadsell. Progressive neural networks. *arXiv preprint arXiv:1606.04671*, 2016.
- Tim Salimans and Jonathan Ho. Progressive distillation for fast sampling of diffusion models. In *International Conference on Learning Representations*, 2021.
- Simone Scardapane, Danilo Comminiello, Amir Hussain, and Aurelio Uncini. Group sparse regularization for deep neural networks. *Neurocomputing*, 241:81–89, jun 2017. doi: 10.1016/j.neucom.2017.02.029. URL <https://doi.org/10.1016%2Fj.neucom.2017.02.029>.
- Hanul Shin, Jung Kwon Lee, Jaehong Kim, and Jiwon Kim. Continual learning with deep generative replay. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- James Seale Smith, Yen-Chang Hsu, Lingyu Zhang, Ting Hua, Zsolt Kira, Yilin Shen, and Hongxia Jin. Continual diffusion: Continual customization of text-to-image diffusion with C-LoRA. *arXiv preprint arXiv:2304.06027*, 2023.
- Jascha Sohl-Dickstein, Eric Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep unsupervised learning using nonequilibrium thermodynamics. In *International Conference on Machine Learning*, pp. 2256–2265. PMLR, 2015.
- Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models. In *International Conference on Learning Representations*, 2020a.
- Yang Song and Stefano Ermon. Generative modeling by estimating gradients of the data distribution. *Advances in Neural Information Processing Systems*, 32, 2019.
- Yang Song, Jascha Sohl-Dickstein, Diederik P Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. In *International Conference on Learning Representations*, 2020b.

- Yang Song, Prafulla Dhariwal, Mark Chen, and Ilya Sutskever. Consistency models. In *Proceedings of the 40th International Conference on Machine Learning*, 2023.
- Gido M. van de Ven, Hava T. Siegelmann, and Andreas S. Tolias. Brain-inspired replay for continual learning with artificial neural networks. *Nature Communications*, 11:4069, 2020.
- Gido M van de Ven, Tinne Tuytelaars, and Andreas S Tolias. Three types of incremental learning. *Nature Machine Intelligence*, 4(12):1185–1197, 2022.
- Mitchell Wortsman, Vivek Ramanujan, Rosanne Liu, Aniruddha Kembhavi, Mohammad Rastegari, Jason Yosinski, and Ali Farhadi. Supermasks in superposition. *Advances in Neural Information Processing Systems*, 33:15173–15184, 2020.
- Han Xiao, Kashif Rasul, and Roland Vollgraf. Fashion-MNIST: a novel image dataset for benchmarking machine learning algorithms. *arXiv preprint arXiv:1708.07747*, 2017.
- Jaehong Yoon, Eunho Yang, Jeongtae Lee, and Sung Ju Hwang. Lifelong learning with dynamically expandable networks. *arXiv preprint arXiv:1708.01547*, 2017.
- Michał Zając, Kamil Deja, Anna Kuzina, Jakub M Tomczak, Tomasz Trzciński, Florian Shkurti, and Piotr Miłoś. Exploring continual learning of diffusion models. *arXiv preprint arXiv:2303.15342*, 2023.
- Friedemann Zenke, Ben Poole, and Surya Ganguli. Continual learning through synaptic intelligence. In Doina Precup and Yee Whye Teh (eds.), *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pp. 3987–3995. PMLR, 06–11 Aug 2017. URL <https://proceedings.mlr.press/v70/zenke17a.html>.

A EXTRA DETAILS ON THE EXPERIMENTS

Code is publicly available at https://github.com/Atenrev/diffusion_continual_learning.

A.1 MODEL ARCHITECTURES

A.1.1 DIFFUSION MODEL

For the experiments on Fashion-MNIST, we employ a custom 2D U-Net based on the one used in [Ho et al. \(2020\)](#). We set the input size to 32×32 pixels, with a single input channel and a single output channel. We use four decreasing feature map resolutions (32×32 to 4×4) with progressively increasing output channel dimensions: 64, 128, 256, and 512, respectively, each block containing one layer. The normalization is applied with 32 groups, and the model has two convolutional residual blocks per resolution level and self-attention blocks between the convolutional blocks at resolutions 8×8 and 4×4 . The total amount of parameters is 42.9 million.

For the experiments in CIFAR-10, we use the same architecture used in [Ho et al. \(2020\)](#) for the same dataset without any customization.

A.1.2 CONTINUALLY TRAINED CLASSIFIER

Table A.1: Architecture used for the continually trained classifier.

Layer	Type	Filters	Kernel Size	Activation
1	Conv2D	32	3x3	ReLU
2	Conv2D	32	3x3	ReLU
3	MaxPool2D	-	2x2	-
4	Dropout (p=0.25)	-	-	-
5	Conv2D	64	3x3	ReLU
6	Conv2D	128	3x3	ReLU
7	MaxPool2D	-	2x2	-
8	Dropout (p=0.25)	-	-	-
9	Conv2D	128	1x1	ReLU
10	AdaptiveMaxPool2D	1	-	-
11	Dropout (p=0.25)	-	-	-
12	Linear	10	-	-

A.1.3 CLASSIFIER USED FOR COMPUTING THE KLD METRIC

In the implementation of the KLD metric we use a ResNet18, which we had trained to classify the ten classes of Fashion-MNIST and CIFAR-10. For the training of this model on Fashion-MNIST, we split the training set into a training set consisting of 50,000 samples and a validation set consisting of 10,000. On CIFAR-10, we split the training set into a training set consisting of 45,000 samples and a validation set consisting of 5,000. We evaluated the model’s accuracy on the validation set at the end of each epoch. Upon completing the training process, we chose the model from the epoch where it achieved the best validation accuracy. We applied the following augmentations, hyperparameters, and architectural changes:

Augmentations:

- Randomly resized the images to 32x32 pixels.
- Applied random horizontal flips to the images.
- Applied random rotations up to 10 degrees.
- Applied random affine transformations with a translation of up to 0.1 in both directions.
- Added Gaussian noise with a mean of 0 and a standard deviation of 0.1 to the images.

Hyperparameters:

- Batch size: 64
- Number of training epochs: 100
- Learning rate: 0.0005

Architectural change to ResNet18:

- Replaced the final fully connected layer with a new linear layer with the number of output classes set to 10.

A.2 GENERATIVE REPLAY IN THE CLASSIFIER

We continually train a classifier with the architecture displayed in Table A.1 along with the diffusion model. This provides an additional way to evaluate the performance of the diffusion model when used as a generator in generative replay of a classifier (Shin et al., 2017). If the generator was ideal and approximated the real data distribution accurately, generative replay would be close to performing experience replay with an unlimited replay buffer.

For tasks $T_i, i > 1$, we train the classifier S_i on samples from the current task T_i , interleaved with synthetic samples generated by $\epsilon_{\hat{\theta}_{i-1}}$, a copy of the knowledge-accumulating generator after training on the previous task. To generate the replay samples to train the classifier, we always use 10 DDIM steps on Fashion-MNIST and 20 DDIM steps on CIFAR-10. Since we use an unconditional diffusion model, we employ soft labelling, following van de Ven et al. (2020), which has shown greater effectiveness than hard labelling. These soft labels result from running the synthetic samples through the previous classifier S_{i-1} . Our objective is to minimize two distinct losses: one for the current task and one for previous tasks. The first loss $L_{\text{curr}}^{\text{clf}}$ is the usual cross-entropy classification defined as:

$$L_{\text{curr}}^{\text{clf}} = \mathbb{E}_{(x_{\text{curr}}, y) \sim D_i} \left[- \sum_{c=1}^{N_{\text{classes}}} y_c \log p_{\theta}(Y = c | x_{\text{curr}}) \right]. \quad (11)$$

The second loss is the distillation loss that makes use of soft labels defined as:

$$L_{\text{replay}}^{\text{clf}} = \mathbb{E}_{x_{\text{prev}} \sim \epsilon_{\hat{\theta}_{i-1}}, \tilde{y} \sim S_{i-1}} \left[- R^2 \sum_{c=1}^{N_{\text{classes}}} \tilde{y}_c \log p_{\theta}^R(Y = c | x_{\text{prev}}) \right] \quad (12)$$

where x_{prev} is the input data for the current task labelled with a hard target y , x_{prev} is the input data for previous tasks generated by the previous generator, $\tilde{y}$ are the soft targets for x_{prev} , and R is a temperature parameter that we set to 2.

The final objective is the joint loss that combines both with a ratio of relative importance $r = \frac{1}{i}$, with i the number of tasks so far, that gives L_{replay} an importance proportional to the number of tasks seen so far:

$$L_{\text{classifier}} = r L_{\text{curr}}^{\text{clf}} + (1 - r) L_{\text{replay}}^{\text{clf}} \quad (13)$$

The experimental setup is discussed in Appendix A.4.

A.3 ADDITIONAL DISTILLATION APPROACHES

The implementation of the distillation with current data is based on Equation (10), except that in L_{distil} in Equation (9), x_t is equal to the same noisy data from the current batch used in the standard training loss $x_t : \sqrt{\alpha_t} \cdot x_0 + \sqrt{1 - \alpha_t} \cdot \epsilon$. Distillation with Gaussian noise is the same, except that now Gaussian noise is used as the “noisy data input”, such that $x_t \sim \mathcal{N}(0, I)$ for all timesteps.

A.4 EXPERIMENTAL SETUP

In our experiment, we continually train a diffusion model along a classifier. When training on a new task, a copy of the diffusion model from previous tasks $\epsilon_{\hat{\theta}_{i-1}}$ generates replay samples for the training of both the diffusion model (only on generative methods) and the classifier. To generate the replay samples to train the diffusion model, for all experiments reported in the main text, we use 2 DDIM steps on Fashion-MNIST and 10 DDIM steps on CIFAR-10. In the Appendix B, we additionally report results for experiments in which we use 2, 10 or 100 DDIM steps to generate the replay samples to train the diffusion model. The replay samples to train the classifier are generated using 10 DDIM steps on Fashion-MNIST or 20 DDIM steps on CIFAR-10, see Appendix A.2. No augmentations are applied to any of the samples. We standardize the ground truth data to fall within the interval of $[-0.5, 0.5]$ and resize it to 32×32 .

For each task, the diffusion model is trained for 100 epochs (9,400 gradient updates) on Fashion-MNIST and 200 epochs (31,400 gradient updates) on CIFAR-10. The classifier is trained for 20 epochs (1,800 gradient updates). On Fashion-MNIST, the batch size is set to 128 for the first task and 256 for the subsequent ones to accommodate the synthetic samples (each batch then contains 128 samples from the current task and 128 replayed samples). On CIFAR-10, the batch size is set to 64 for the first task and 128 for the subsequent ones. For distillation with current

data the batch size is always 128 and 64 for Fashion-MNIST and CIFAR-10 respectively. We train both models using Adam (Kingma & Ba, 2014) as the optimizer. For the diffusion model, we use a learning rate of 2×10^{-4} , while for the classifier we use 1×10^{-3} . We implement all the strategies using the Avalanche library (Lomonaco et al., 2021).

In order to find the best λ for each distillation approach, we sweep over different sets of lambda values based on the observed loss magnitudes. We sweep on Fashion-MNIST across 3 random seeds over the values of $\{0.1, 0.25, 0.5, 0.75, 1.0, 1.25, 1.5, 1.75, 2.0, 2.25, 2.5, 2.75, 3.0\}$ for generative distillation, $\{1, 4, 8, 12, 16, 20, 24, 28\}$ for distillation with Gaussian noise, and $\{0.1, 0.25, 0.5, 0.75, 1.0, 1.25, 1.5, 1.75\}$ for distillation with current data. We select the lambda that yields the best FID value for the comparisons and use it for both Fashion-MNIST and CIFAR-10.

A.5 EVALUATION

The models are evaluated after finishing the training on each task. For these evaluations, we only use the part of the test set corresponding to all the classes seen so far. To compute the Fréchet Inception distance (FID), we generate 10,000 images with the diffusion model using 10 DDIM steps. We use these same 10,000 images to compute the Kullback-Leibler divergence (KLD) between the ground truth class frequency distribution and the generated class frequency distribution. Since we are using an unconditional diffusion model architecture, we use a ResNet18 (He et al., 2016) trained on Fashion-MNIST or on CIFAR-10 to label the generated samples; see Appendix A.1.3 for further details on the training of this classifier. As an additional metric, we report the accuracy of the continually trained classifier with generative replay (see Appendix A.2 for details).

To visualize example images sampled from the trained diffusion models, we always use 10 DDIM steps on Fashion-MNIST and 20 DDIM steps on CIFAR-10.

B EXTRA EXPERIMENTS

B.1 SAMPLES FROM A MODEL TRAINED JOINTLY ON ALL DATA

To motivate the choice of the number of teacher DDIM steps, we trained a diffusion model in Fashion-MNIST and CIFAR-10 with an independently and identically distributed stream of data for 100 epochs (equivalent to 46,900 gradient updates) and 200 epochs (equivalent to 78,200 gradient updates) respectively. We set the batch size to 128 and the learning rate to 2×10^{-4} . With this model, we used DDIM to generate images using 2, 10 and 100 steps. For comparison, we also sampled images using DDPM.

Fashion-MNIST As illustrated in Figure B.1, employing 2 DDIM steps resulted in a distinguishable but smooth approximation of the samples. Using 10 DDIM steps provided decent looking samples; increasing the DDIM step count to 100 did not seem to enhance the visual quality of the sampled images much.

CIFAR-10 In contrast with the results obtained with the model trained on Fashion-MNIST, the model trained on CIFAR-10 produced noisy and undistinguishable samples when using 2 DDIM steps as shown in Figure B.1. When the number of DDIM steps is increased to 10, the objects become recognizable despite being a bit smooth. With 100 DDIM steps or when using DDPM, the images yielded by the model become sharper and more detailed.

B.2 COMPUTATIONAL AND MEMORY OVERHEAD

Compared with generative replay, generative distillation should increase the computational costs by just one forward pass. This extra forward pass is needed to compute $\epsilon_{\hat{\theta}_{i-1}}(x_t, t)$ in Equation (9). To verify this in practice, here we compare the empirical wall time per iteration for finetuning, generative replay and generative distillation. To obtain the wall time per iteration for each method, we take the average over 1000 training steps on an NVIDIA GeForce 1080Ti GPU. As in our main experiments, when generating replay data in generative replay and generative distillation, we use 2 DDIM teacher steps with Fashion MNIST and 10 DDIM teacher steps with CIFAR-10. As shown in Table B.1, when comparing generative distillation with generative replay, the relative increase in wall time on Fashion MNIST (1.13×) and CIFAR-10 (1.06×) is moderate, especially considering the substantially lower FID achieved on Fashion MNIST (9.01×) and on CIFAR-10 (2.28×) (refer to Table 1 in the main text).

We also report the memory requirements of each method. For finetuning, there is no need to store either samples or model copies; only the parameters of the model that is being trained need to be kept in memory (for Fashion MNIST: 42,855,873 parameters; for CIFAR-10: 35,746,307 parameters). For generative replay and generative distillation, there is no need to store a buffer of synthetic samples (since we generate the samples to be replayed at each iteration), but

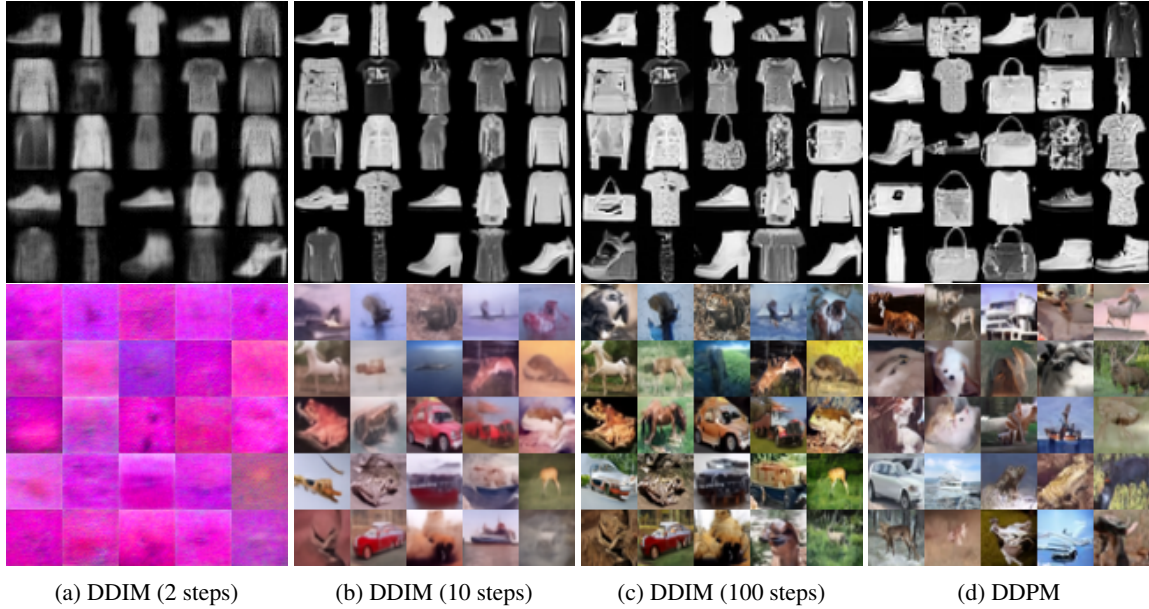


Figure B.1: **Samples from a jointly trained model on Fashion-MNIST (first row) and CIFAR-10 (second row) with different sampling methods.** Relatively smooth samples are generated with 2 DDIM steps, minor artefacts are present with 100 DDIM steps.

Table B.1: **Computational overhead for different approaches.** Reported is the empirical wall-time (in ms) per iteration (mean $\pm$ standard deviation over 1000 iterations).

Strategy	Fashion MNIST	CIFAR-10
	$\downarrow$ wall-time (ms/it)	$\downarrow$ wall-time (ms/it)
Finetune	253.3 $\pm$ 8.8	383.4 $\pm$ 95.3
Generative replay	664.3 $\pm$ 3.8	1881.8 $\pm$ 42.9
Generative distillation	747.9 $\pm$ 3.6	1992.7 $\pm$ 47.6

for both approaches one extra model copy needs to be stored (the previous task model $\epsilon_{\hat{\theta}_{i-1}}$). Therefore, generative replay and generative distillation have the same memory requirement, double that of finetuning (for Fashion MNIST: 85,711,746 parameters; for CIFAR-10: 71,492,614 parameters).

B.3 COMPARING GENERATIVE DISTILLATION WITH EXPERIENCE REPLAY

We include additional experiments where we compare the performance of generative distillation with experience replay using various memory buffer sizes. We consider memory buffers with a total capacity of 10, 100, and 1000 samples. After each task, the capacity of the buffer is equally divided over all previous tasks (i.e., with a total capacity of ten samples, after the first task ten samples per task are stored, after the second task five samples per task are stored, and so on). The samples to be added to the memory buffer are randomly selected from the last task’s training data, and the samples to be discarded from the buffer (to make place for the samples from the last task) are also selected at random. During training, in each iteration, the samples to be replayed are randomly sampled from the buffer. Similar to our implementation of generative replay, the size of the mini-batch of replay data matches that of the current task mini-batch (if the memory buffer size is smaller than the mini-batch size, all samples from the memory buffer are replayed). We compute a separate loss on the data of the current task and on the replayed data (using Equation (4) for both), and we combine these two losses as in Equation (8).

In Table B.2, the results indicate that increasing the buffer size generally improves the performance of experience replay. On Fashion MNIST, using a buffer with total capacity of 1000 achieves comparable performance to generative distillation in terms of FID and KLD. On CIFAR-10, using a buffer of 100 samples results in comparable performance

Table B.2: **Quantitative comparison of generative distillation and experience replay.** The FID and KLD are computed after training on all tasks. Reported for each metric is mean $\pm$ standard error over 3 random seeds. The buffer size indicates the total number of samples from previous tasks that is allowed to be kept in the memory buffer.

Strategy	Buffer size	Fashion MNIST		CIFAR-10	
		$\downarrow$ FID	$\downarrow$ KLD	$\downarrow$ FID	$\downarrow$ KLD
Joint (<i>upper target</i>)	-	15.5 ± 1.3	0.08 ± 0.02	31.9 ± 2.9	0.12 ± 0.02
Finetune (<i>lower target</i>)	-	68.8 ± 7.0	4.82 ± 1.42	55.8 ± 2.6	1.14 ± 0.10
Experience replay	10	56.2 ± 1.1	0.72 ± 0.09	62.0 ± 4.9	0.48 ± 0.05
Experience replay	100	33.3 ± 2.9	0.33 ± 0.04	42.3 ± 0.9	0.25 ± 0.03
Experience replay	1000	21.6 ± 0.9	0.28 ± 0.01	49.6 ± 5.4	0.48 ± 0.05
Generative distillation	-	23.5 ± 1.1	0.25 ± 0.10	42.3 ± 4.7	0.15 ± 0.01

Table B.3: **Comparison of generative distillation and experience replay with a continually trained classifier.** The classifier is trained on the data of the current task along with replay samples from the continually trained diffusion model. Reported is mean $\pm$ standard error over 3 seeds. The buffer size indicates the total number of samples from previous tasks that is allowed to be kept in the memory buffer for the training of the diffusion model.

Strategy	Buffer size	Fashion MNIST	CIFAR-10
		$\uparrow$ ACC	$\uparrow$ ACC
Joint (<i>upper target</i>)	-	0.92 ± 0.00	0.77 ± 0.03
Finetune (<i>lower target</i>)	-	0.17 ± 0.03	0.19 ± 0.00
Experience replay	10	0.60 ± 0.02	0.12 ± 0.01
Experience replay	100	0.79 ± 0.01	0.12 ± 0.01
Experience replay	1000	0.76 ± 0.03	0.13 ± 0.01
Generative distillation	-	0.72 ± 0.03	0.31 ± 0.03

to generative distillation in terms of FID, but not in terms of KLD. Moreover, somewhat surprisingly, increasing the buffer size from 100 to 1000 samples does not improve the FID or KLD of experience replay on CIFAR-10.

In Table B.3, we further compare generative distillation and experience replay through the performance of a classifier that is continually trained with generative replay produced by the continually trained diffusion models (see Section 4.2.3). On Fashion MNIST, a diffusion model continually trained with experience replay using a buffer size of 100 samples already provides quite good samples to train a classifier. However, on the more complex dataset CIFAR-10, the diffusion model trained with experience replay fails to generate samples that can train a classifier well even with a buffer size of 1000 samples.

B.4 ADDITIONAL QUANTITATIVE RESULTS

To report the full results of the additional experiments described in Section 4.3, we provide Tables B.4 and B.5 (quantitative results) and Figure B.2 (graph of quality metrics throughout the continual learning process).

Table B.4: **Full comparison results among different approaches on Fashion-MNIST.** Reported for each metric is the mean $\pm$ standard error over 3 random seeds. The number of generation steps used by the generator from the previous task to train the current task model (if any) is indicated in the *Teacher steps* column. All models used 10 DDIM steps to generate images for the classifier during generative replay.

Strategy	Teacher steps	λ	$\downarrow$ FID	$\downarrow$ KLD	$\uparrow$ ACC
Joint (classifier)	-	-	-	-	0.92 ± 0.00
Finetune (classifier)	-	-	-	-	0.17 ± 0.03
Joint (generator)	-	-	15.49 ± 1.30	0.08 ± 0.02	-
Finetune (generator)	-	-	68.76 ± 7.00	4.82 ± 1.42	-
Distil. w/ Gaussian noise	-	8	38.87 ± 2.23	0.85 ± 0.05	0.63 ± 0.04
Distil. w/ current data	-	1.25	65.01 ± 3.31	0.37 ± 0.09	0.35 ± 0.01
Generative replay	2	-	211.79 ± 6.92	3.39 ± 0.30	0.50 ± 0.04
Generative distillation	2	0.75	23.53 ± 1.06	0.25 ± 0.10	0.72 ± 0.03
Generative replay	10	-	121.42 ± 13.04	0.98 ± 0.19	0.58 ± 0.04
Generative distillation	10	0.75	20.29 ± 3.07	0.13 ± 0.04	0.79 ± 0.01
Generative replay	100	-	57.54 ± 8.61	0.61 ± 0.21	0.66 ± 0.02
Generative distillation	100	0.75	23.69 ± 3.42	0.12 ± 0.04	0.80 ± 0.01

Table B.5: **Full comparison results among different approaches on CIFAR-10.** Reported for each metric is the mean $\pm$ standard error over 3 random seeds. The number of generation steps used by the generator from the previous task to train the current task model (if any) is indicated in the *Teacher steps* column. All models used 10 DDIM steps to generate images for the classifier during generative replay.

Strategy	Teacher steps	λ	$\downarrow$ FID	$\downarrow$ KLD	$\uparrow$ ACC
Joint (classifier)	-	-	-	-	0.77 ± 0.00
Finetune (classifier)	-	-	-	-	0.19 ± 0.00
Joint (generator)	-	-	31.88 ± 2.89	0.12 ± 0.02	-
Finetune (generator)	-	-	55.77 ± 2.56	1.14 ± 0.10	-
Distil. w/ Gaussian noise	-	8	51.35 ± 2.95	0.75 ± 0.05	0.30 ± 0.00
Distil. w/ current data	-	1.25	62.09 ± 2.49	0.49 ± 0.09	0.23 ± 0.02
Generative replay	2	-	166.69 ± 15.95	1.45 ± 0.45	0.27 ± 0.02
Generative distillation	2	0.75	48.82 ± 2.51	0.69 ± 0.02	0.33 ± 0.02
Generative replay	10	-	96.39 ± 5.25	0.74 ± 0.14	0.26 ± 0.01
Generative distillation	10	0.75	42.34 ± 4.74	0.15 ± 0.01	0.31 ± 0.03
Generative replay	100	-	66.47 ± 7.39	0.43 ± 0.07	0.28 ± 0.02
Generative distillation	100	0.75	30.53 ± 1.29	0.27 ± 0.05	0.35 ± 0.03

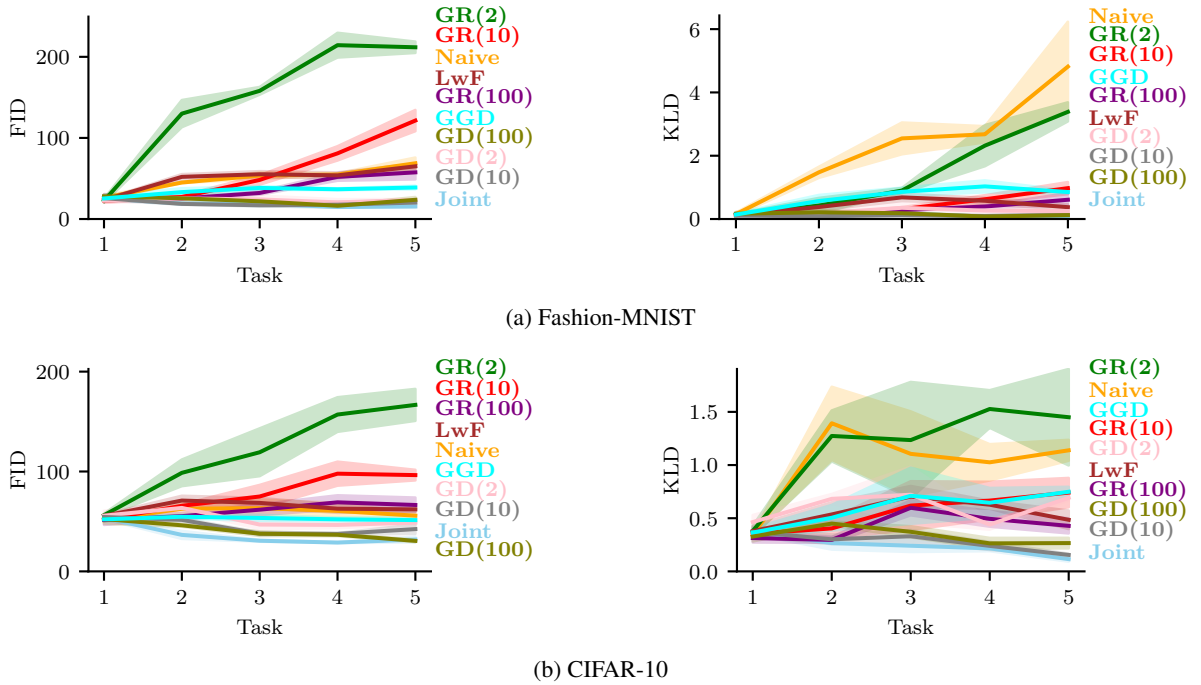


Figure B.2: **Quality of the diffusion model throughout the continual learning process (with additional approaches).** Each point represents a metric achieved after training in a specific task. Displayed are the means $\pm$ standard errors over 3 random seeds. Joint: training using data from all tasks seen so far (upper target), Naive: fine-tuning on the current task (lower target), GR(n): generative replay with a teacher using n DDIM steps, LwF: distillation with current data, GGD: distillation with Gaussian noise, GD: generative distillation.

B.5 SAMPLES THROUGHOUT THE CONTINUAL LEARNING PROCESS

In this section, we display samples generated by the diffusion model throughout the continual learning process following the different approaches we have discussed. This allows the reader to qualitatively evaluate the diffusion model’s performance under each approach and throughout their continual training. The samples were obtained using a consistent representative random seed across all tasks and approaches. Note that we could not make sure that all the experiments were executed on the same hardware, thus some runs may vary slightly (refer to <https://pytorch.org/docs/stable/notes/randomness.html>).

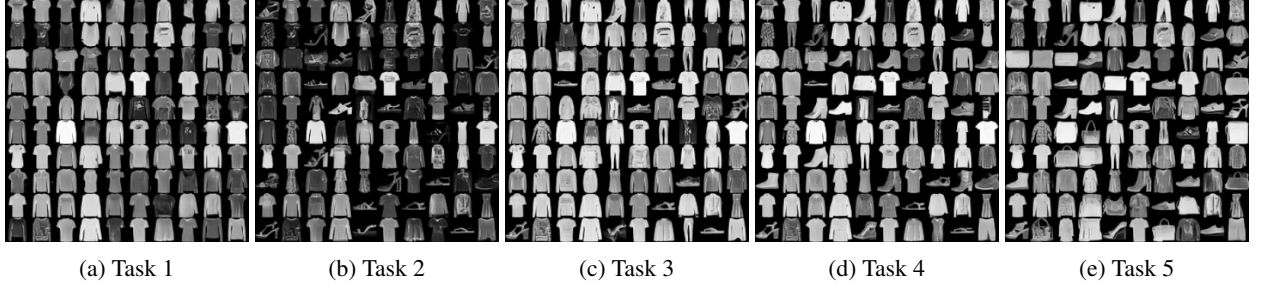


Figure B.3: **Samples for joint training on Fashion-MNIST.** This can be interpreted as an upper target.

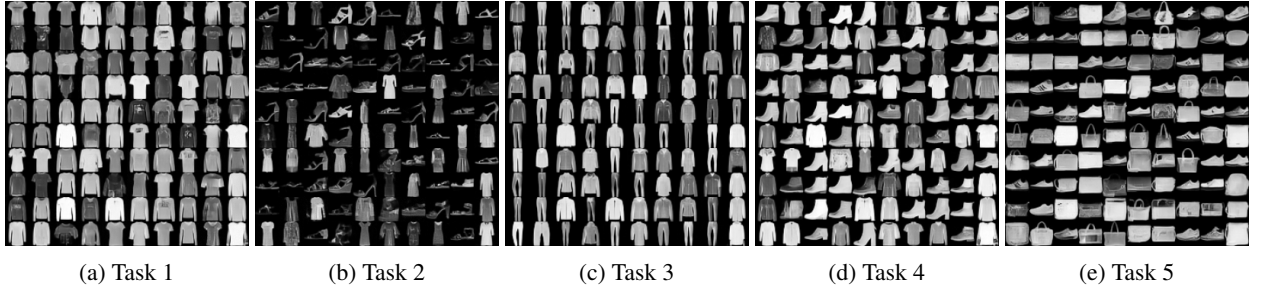


Figure B.4: **Samples for finetune on Fashion-MNIST.** This can be interpreted as a lower target.

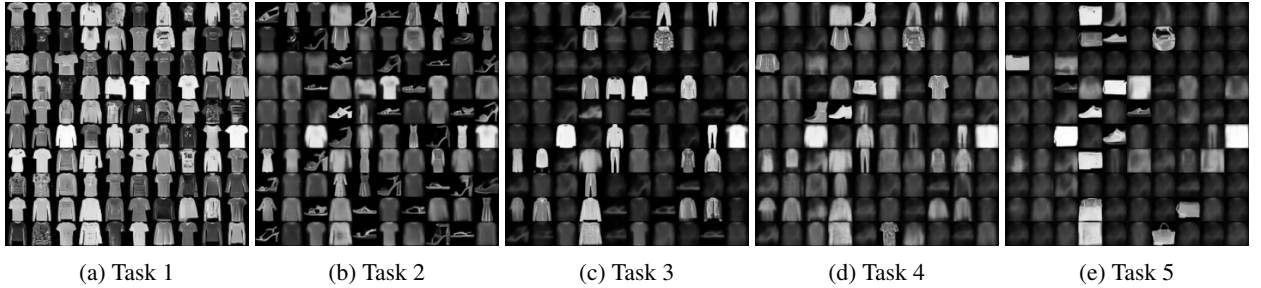


Figure B.5: **Samples for generative replay (2 teacher DDIM steps) on Fashion-MNIST.** Standard generative replay causes a catastrophic degradation of the denoising capabilities of the diffusion model for samples from previous tasks.

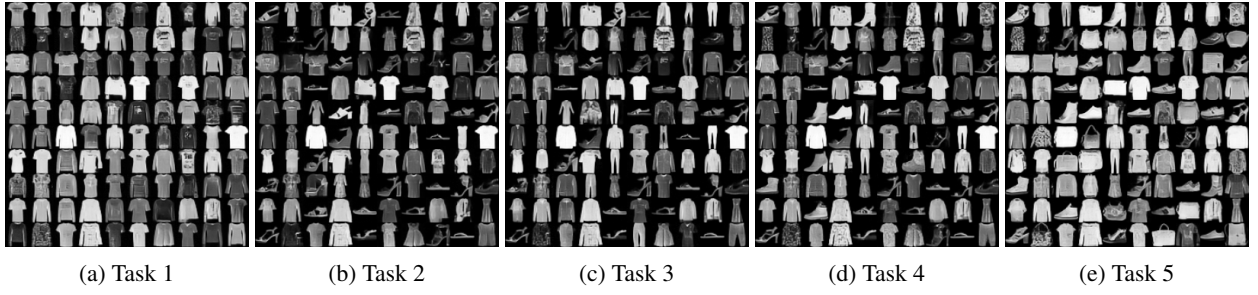


Figure B.6: **Samples for generative distillation (2 teacher DDIM steps) on Fashion-MNIST.** Generative distillation markedly improves the quality with only a moderate increase in the computational cost.

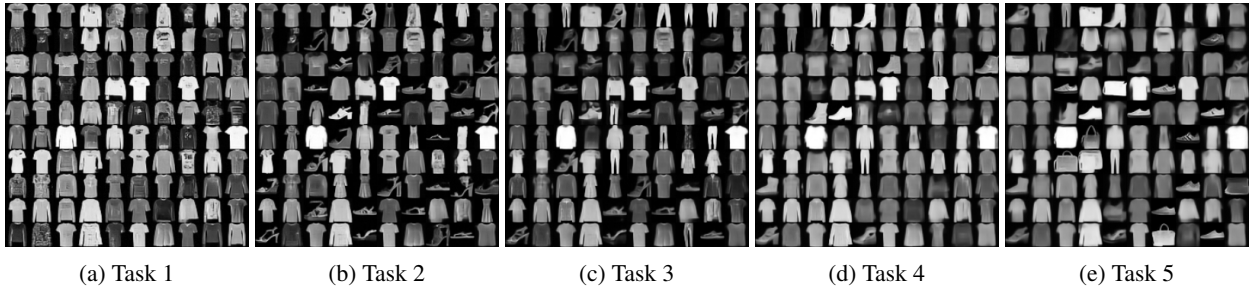


Figure B.7: **Samples for generative replay (10 teacher DDIM steps) on Fashion-MNIST.** Increasing the number of teacher DDIM steps reduces but does not solve the catastrophic loss in denoising capabilities.

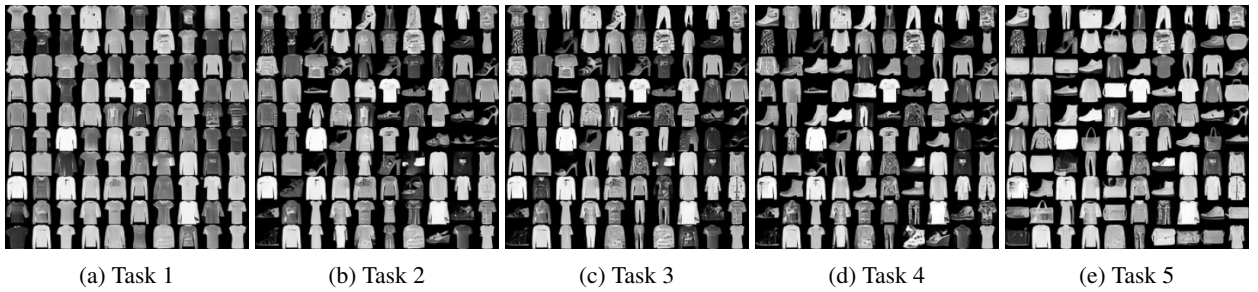


Figure B.8: **Samples for generative distillation (10 teacher DDIM steps) on Fashion-MNIST.** Generative distillation markedly improves the quality with only a moderate increase in the computational cost.

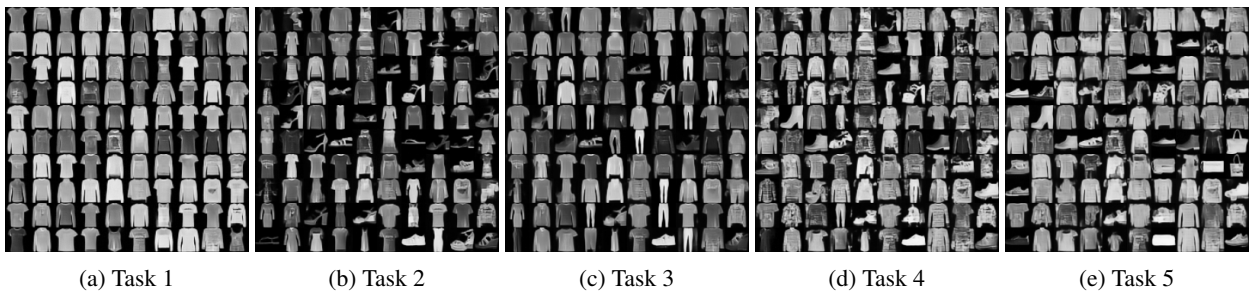


Figure B.9: **Samples for generative replay (100 teacher DDIM steps) on Fashion-MNIST.** Increasing the number of teacher DDIM steps reduces but does not solve the catastrophic loss in denoising capabilities.

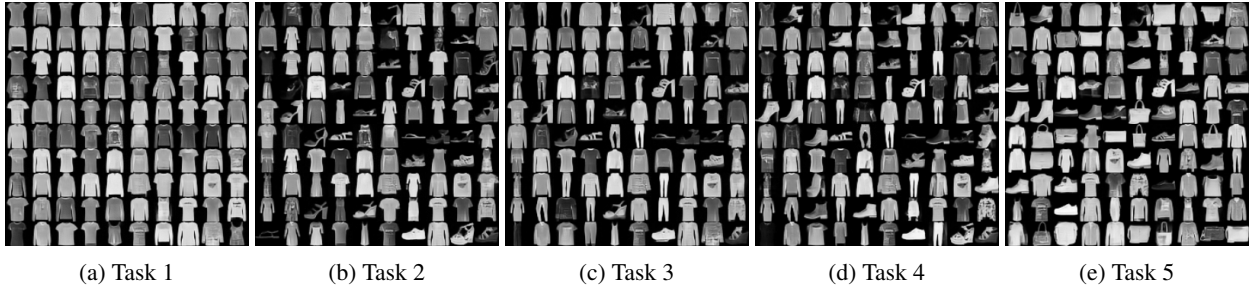


Figure B.10: **Samples for generative distillation (100 teacher DDIM steps) on Fashion-MNIST.** Generative distillation markedly improves the quality with only a moderate increase in the computational cost.

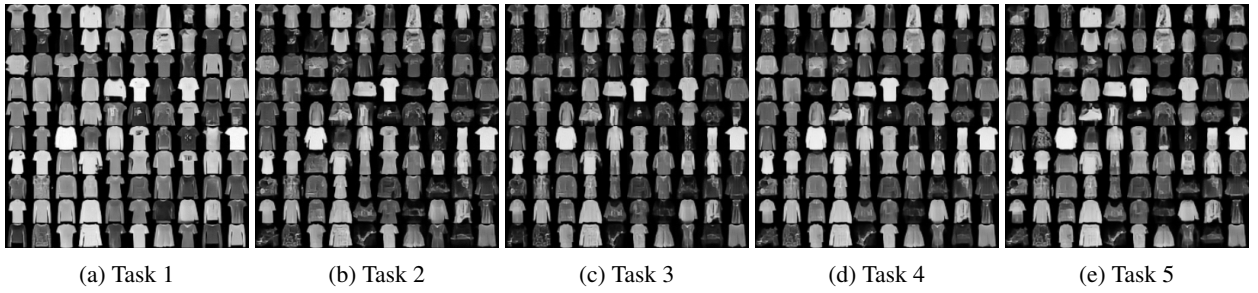


Figure B.11: **Samples for LwF-like distillation on Fashion-MNIST.** The LwF-like approach improves over naïve generative replay but still has an image quality degradation over the continual learning process.

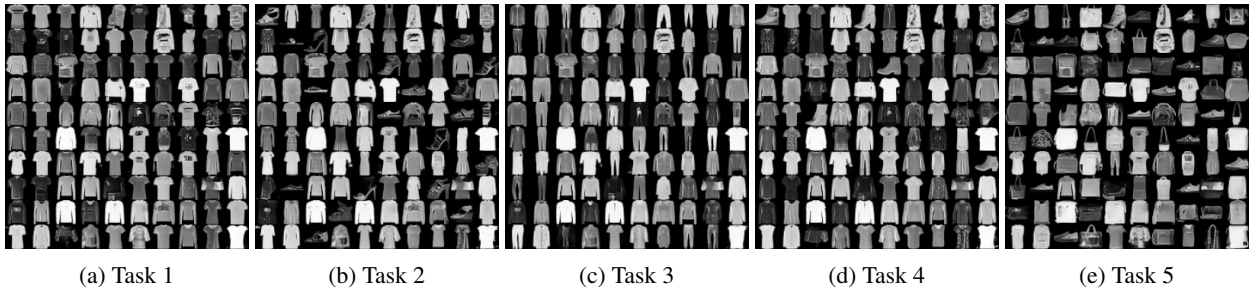


Figure B.12: **Samples for distillation with Gaussian noise on Fashion-MNIST.** Similar to distilling with current data, distilling with noise preserves knowledge of classes similar to the ones in the last task learned.



Figure B.13: **Samples for joint training on CIFAR-10.** This can be interpreted as an upper target.



Figure B.14: **Samples for finetune on CIFAR-10.** This can be interpreted as a lower target.



Figure B.15: **Samples for generative replay (2 teacher DDIM steps) on CIFAR-10.** Standard generative replay causes a catastrophic degradation of the denoising capabilities of the diffusion model for samples from previous tasks.



Figure B.16: **Samples for generative distillation (2 teacher DDIM steps) on CIFAR-10.** Generative distillation markedly improves the quality with only a moderate increase in the computational cost.



Figure B.17: **Samples for generative replay (10 teacher DDIM steps) on CIFAR-10.** Increasing the number of teacher DDIM steps reduces but does not solve the catastrophic loss in denoising capabilities.



Figure B.18: **Samples for generative distillation (10 teacher DDIM steps) on CIFAR-10.** Generative distillation markedly improves the quality with only a moderate increase in the computational cost.



Figure B.19: **Samples for generative replay (100 teacher DDIM steps) on CIFAR-10.** Increasing the number of teacher DDIM steps reduces but does not solve the catastrophic loss in denoising capabilities.



Figure B.20: **Samples for generative distillation (100 teacher DDIM steps) on CIFAR-10.** Generative distillation markedly improves the quality with only a moderate increase in the computational cost.



Figure B.21: **Samples for LwF-like distillation on CIFAR-10.** The LwF-like approach improves over naïve generative replay but still has an image quality degradation over the continual learning process.



Figure B.22: **Samples for distillation with Gaussian noise on CIFAR-10.** Similar to distilling with current data, distilling with noise preserves knowledge of classes similar to the ones in the last task learned.